CASE STUDY

Conflict-Free Replicated Data Types

Enabling Consistency Without Coordination in Distributed Systems

Submitted by: Santulan Deshmukh (Roll No. 151)	**Programme:** B.Tech – Information Technology
Institution: Yeshwantrao Chavan College of Engineering, Nagpur	**Domain:** Distributed Systems & Data Consistency

1. Executive Summary

This case study examines Conflict-Free Replicated Data Types (CRDTs) — a mathematically rigorous class of data structures designed to solve one of the hardest problems in distributed computing: maintaining consistent shared state across multiple nodes without requiring locks, synchronization barriers, or complex conflict resolution logic.

CRDTs achieve this by defining merge operations that are commutative, associative, and idempotent — properties that together guarantee that any two replicas, given the same set of updates in any order, will converge to an identical state. This report explores the theoretical foundation, algorithmic mechanics, a concrete Java implementation of the G-Counter CRDT, practical real-world applications, and an honest analysis of the trade-offs involved.

> **Key Insight**
> CRDTs replace the question 'Who is right when two nodes disagree?' with 'Can we design the data structure so nodes never truly disagree?' The answer is yes — through mathematics.

2. Problem Statement & Motivation

2.1 The Distributed Consistency Challenge

In a distributed system, multiple nodes operate independently and communicate over an unreliable network. When a user updates data on Node A simultaneously with another user updating the same data on Node B, a conflict arises. Traditional approaches handle this in one of three ways:

- Locking / Mutual Exclusion – Only one node may write at a time. This guarantees strong consistency but kills availability and performance.
- Coordination Protocols (2PC, Paxos, Raft) – Nodes vote before committing. Highly reliable, but adds latency and fails if the coordinator is unreachable.

- Last-Write-Wins (LWW) – Accept the most recent timestamp. Simple, but silently discards legitimate updates.

All three approaches sacrifice either availability, performance, or data integrity. CRDTs offer a fourth path: design the data structure so that merging any two replicas is always correct — no coordinator, no locks, no data loss.

2.2 The CAP Theorem Context

The CAP Theorem (Brewer, 2000) states that a distributed system can guarantee at most two of: Consistency, Availability, and Partition Tolerance. CRDTs align with the AP corner of this triangle — they prioritize Availability and Partition Tolerance, and deliver Eventual Consistency in place of strong consistency. This is an acceptable and often preferred trade-off for collaborative and real-time applications.

Property	Traditional Locking	CRDT Approach
Consistency	Strong (linearizable)	Eventual (convergent)
Availability	Low (blocked by locks)	High (always writable)
Partition Tolerance	Low	High
Concurrency Support	Sequential only	Fully concurrent
Conflict Resolution	Manual / LWW	Automatic (math-based)
Offline Support	None	Full support

3. Technical Background

3.1 What is a CRDT?

A Conflict-Free Replicated Data Type is a data structure augmented with a merge function that satisfies three mathematical properties:

Property	Definition & Guarantee
Commutativity	merge(A, B) = merge(B, A) — Order of merging does not matter
Associativity	merge(A, merge(B, C)) = merge(merge(A, B), C) — Grouping does not matter
Idempotency	merge(A, A) = A — Merging a state with itself produces no change

These three properties together form a Join-Semilattice — a mathematical structure where any two elements have a unique least upper bound (the merged result). This guarantee is what makes CRDTs provably correct without coordination.

3.2 Two Flavors of CRDTs

Type	Description
State-based CRDTs (CvRDT)	Nodes share their entire state periodically. The merge function combines two states. Simpler to implement; higher bandwidth usage. Example: G-Counter, OR-Set.
Operation-based CRDTs (CmRDT)	Nodes broadcast individual operations (deltas). Requires reliable delivery but uses less bandwidth. Example: Operational Transforms, delta-CRDTs.

4. Algorithm Deep Dive: G-Counter CRDT

4.1 Data Structure Design

The Grow-Only Counter (G-Counter) is the simplest state-based CRDT. It is a foundational building block — more complex CRDTs such as PN-Counters and 2P-Sets are composed from G-Counters.

- Each node i in a cluster of N nodes maintains a vector V of N integers.
- Node i is only permitted to increment V[i] — its own slot.
- The true global count is the sum of all elements: count = Σ V[i] for i in 0..N-1.

4.2 Step-by-Step Algorithm

G-Counter Algorithm — Five Phases
Step 1 — Initialization: Each node initializes its vector to all zeros. e.g., Node A: [0,0] Node B: [0,0]
Step 2 — Local Update: Node i increments only V[i]. No network call is made. No lock is needed.
Step 3 — Dissemination: Periodically, nodes broadcast their full state vector via gossip or direct push.
Step 4 — Merge: On receiving state S_other, node applies: V[i] = max(V[i], S_other[i]) for all i.
Step 5 — Convergence: After all updates propagate, every node's vector is identical. Sum = true count.

4.3 Worked Execution Example

Trace of a two-node G-Counter through concurrent increments and a merge:

Event	Node A State	Node B State
Initial State	[0, 0] → count: 0	[0, 0] → count: 0
Node A increments ×3	[3, 0] → count: 3	[0, 0] → count: 0
Node B increments ×2	[3, 0] → count: 3	[0, 2] → count: 2
Node A sends state to B	[3, 0]	merge([0,2],[3,0]) → [3,2]
Node B sends state to A	merge([3,0],[3,2]) → [3,2]	[3, 2]
Final Converged State	[3, 2] → count: 5	[3, 2] → count: 5

Why No Conflict?

Node A's 3 increments are captured in slot 0; Node B's 2 are in slot 1. They never write to each other's slot. The max() merge picks up the latest value from each slot without overwriting or losing any update. Total = 3 + 2 = 5. Always correct.

5. Implementation Analysis: Java G-Counter

5.1 Core Class Design

The Java implementation models a G-Counter with two nodes, a Swing-based GUI for visualization, and an animated merge operation. Below is the core data structure:

```
class GCounter {
    private int[] counter;  // vector of per-node counts
    private int nodeId;     // this node's index

    public GCounter(int numNodes, int nodeId) {
        this.counter = new int[numNodes];  // init all zeros
        this.nodeId = nodeId;
    }

    // Only increment own slot — CRDT invariant preserved
    public void increment() {
        counter[nodeId]++;
    }

    // Merge: element-wise maximum — commutativity + idempotency guaranteed
    public void merge(GCounter other) {
        for (int i = 0; i < counter.length; i++) {
            counter[i] = Math.max(counter[i], other.counter[i]);
        }
    }

    // Global value = sum of all slots
```

```
    public int getValue() {
        return Arrays.stream(counter).sum();
    }
}
```

5.2 Implementation Highlights

Feature	Implementation Detail
Invariant Enforcement	increment() modifies only counter[nodeId], enforcing the single-slot ownership rule.
Merge Function	Math.max() per index satisfies commutativity, associativity, and idempotency.
Animated Visualization	A Swing JFrame shows per-slot values in real time during merge with 800ms delay.
Thread Safety	Merge animation runs in a new Thread; UI updates dispatched via SwingUtilities.invokeLater().
Value Computation	Arrays.stream().sum() computes global count as the sum of all node slots.

5.3 Output Walkthrough

In the demonstration output shown (Node A: [5,3], Node B: [5,3], Final Value: 8):

- Node A has 5 increments in slot 0, Node B has 3 increments in slot 1.
- After merge, both nodes hold the vector [5, 3].
- The displayed Final Value is 5 + 3 = 8, correctly representing the total distributed increment count.
- Both nodes converge to identical state — eventual consistency achieved.

6. Real-World Applications

6.1 Industry Adoption

System / Company	CRDT Usage
Redis (Enterprise)	CRDT-based counters, sets, and registers for geo-distributed active-active clusters
Apache Cassandra	LWW registers and counters use CRDT principles for replica reconciliation

Riak KV (Basho)	Native CRDT support for Maps, Sets, Counters, Flags, and Registers
Figma	Multiplayer design canvas uses CRDT-like structures for real-time collaboration
Google Docs	Operational transformation (OT) — related approach solving same problem in text editing
SoundCloud / Orbit	CRDTs for distributed message counters and event logs

6.2 Application Domains

- **Text editors, whiteboards, and design tools where multiple users edit simultaneously.:** Collaborative Editing
- **Geo-replicated databases where each datacenter must accept writes independently.:** Distributed Databases
- **Mobile apps that allow edits offline and sync later without conflicts.:** Offline-First Applications
- **Sensor networks where nodes independently track counts and periodically sync.:** IoT Sensor Aggregation
- **Player score tracking across distributed game servers with eventual aggregation.:** Gaming Leaderboards
- **Like counts, view counts, and reaction counters at massive scale (e.g., Facebook).:** Social Media Counters

7. Critical Analysis: Strengths & Limitations

7.1 Strengths

Strength	Explanation
Lock-Free Concurrency	No mutexes or semaphores required. Every node can write at any time independently.
Offline & Partition Tolerance	Nodes continue functioning during network partitions. Updates sync on reconnection.
Mathematical Correctness	Convergence is a formal, provable property — not a best-effort heuristic.
High Availability	No single point of failure. Loss of any node does not block the system.
Automatic Conflict Resolution	Merge semantics are baked into the data type — no application-level conflict code needed.
Horizontal Scalability	Adding more nodes does not require redesigning the merge protocol.

7.2 Limitations & Trade-Offs

Limitation	Explanation
State Size Growth	State-based CRDTs broadcast their full state. With many nodes, bandwidth cost can be significant.
No Strong Consistency	CRDTs guarantee eventual, not strong, consistency. Not suitable for financial transactions.
Limited Data Types	Not all data structures have CRDT variants. Complex business logic may not fit the model.
Tombstone Accumulation	Deletion in sets/maps often uses tombstones that accumulate and require periodic garbage collection.
Complexity of Advanced CRDTs	Multi-value registers and collaborative text CRDTs (e.g., LSEQ, RGA) are significantly more complex.
Eventual ≠ Immediate	Users may see stale values until convergence occurs, which can be unintuitive for some use cases.

8. Conclusion

Conflict-Free Replicated Data Types represent a paradigm shift in distributed systems design. Rather than fighting the fundamental challenges of concurrency and network unreliability with locks and coordination, CRDTs embrace them — encoding correctness directly into the mathematical structure of the data.

The G-Counter, implemented and visualized in this project, is an elegant proof-of-concept: two nodes increment independently, exchange states, apply element-wise maximums, and converge to an identical, correct result — with zero coordination overhead and zero risk of conflict.

As distributed systems grow more geographically dispersed and users increasingly demand offline capability, the relevance of CRDTs will only grow. Systems like Redis Enterprise, Riak, and collaborative tools like Figma have already demonstrated that CRDTs work at production scale. Understanding CRDTs is therefore not merely academic — it is a practical skill for building the next generation of resilient, scalable, and user-friendly distributed applications.

> **Final Takeaway**
> CRDTs prove that in distributed systems, the best conflict resolution is to design data structures where conflicts are mathematically impossible. That is not a workaround — it is elegant engineering.

9. References

[1] M. Shapiro, N. Preguiça, C. Baquero, and M. Zawirski, *"A Comprehensive Study of Convergent and Commutative Replicated Data Types,"* Research Report RR-7506, INRIA, 2011.

[2] E. Brewer, *"Towards Robust Distributed Systems,"* ACM Symposium on Principles of Distributed Computing (PODC), 2000.

[3] Riak Documentation, *"CRDTs in Riak,"* Basho Technologies. Available: https://docs.riak.com/riak/kv/latest/developing/data-types/

[4] Redis Documentation, *"Active-Active Geo-Distribution,"* Redis Enterprise. Available: https://redis.com/redis-enterprise/technology/active-active-geo-distribution/